Time Series Analysis

Part 1: Lecture 1

Time Series Analysis in R

Key reference

Hyndman, R. J. & Athanasopoulos, G. (2021) *Forecasting: principles and practice*, 3rd edition

OTexts.org/fpp3/

- Free and online
- Data sets in associated R packages
- R code for examples

CRAN Task View

https://cran.r-project.org/web/views/TimeSeries.html

CRAN Task View: Time Series Analysis

Maintainer:	Rob J Hyndman, Rebecca Killick
Contact:	Rob.Hyndman at monash.edu
Version:	2023-07-26
URL:	https://CRAN.R-project.org/view=TimeSeries
Source:	https://github.com/cran-task-views/TimeSeries/
Contributions:	Suggestions and improvements for this task view are very welcome and can be made through issues or pull requests on GitHub or via e-mail to the maintainer address. For further details see the Contributing guide.
Citation:	Rob J Hyndman, Rebecca Killick (2023). CRAN Task View: Time Series Analysis. Version 2023-07-26. URL https://CRAN.R-project.org/view=TimeSeries.
Installation:	The packages from this task view can be installed automatically using the ctv package. For example, `ctv::install.views("TimeSeries", coreOnly = TRUE)` installs all the core packages or `ctv::update.views("TimeSeries")` installs all packages that are not yet installed and up-to-date. See the CRAN Task View Initiative for more details.

Base R ships with a lot of functionality useful for time series, in particular in the stats package. This is complemented by many packages on CRAN, which are briefly summarized below. There is overlap between the tools for time series and those designed for specific domains including Econometrics, Finance and Environmetrics.

The packages in this view can be roughly structured into the following topics. If you think that some package is missing from the list, please let us know, either via e-mail to the maintainer or by submitting an issue or pull request in the GitHub repository linked above.

Packages

Task	timetk	tsibble	feasts	tibbletime
Structure				
Data Structure	tibble (tbl)	tsibble (tbl_ts)	tsibble (tbl_ts)	tibbletime (tbl_time)
Visualization				
Interactive Plots (plotly)	✅	❌	❌	❌
Static Plots (ggplot)	✅	❌	✅	❌
Time Series	✅	❌	✅	❌
Correlation, Seasonality	✅	❌	✅	❌
Data Wrangling				
Time-Based Summarization	✅	❌	❌	✅
Time-Based Filtering	✅	❌	❌	✅
Padding Gaps	✅	✅	❌	❌
Low to High Frequency	✅	❌	❌	❌
Imputation	✅	✅	❌	❌
Sliding / Rolling	✅	✅	❌	✅
Machine Learning				
Time Series Machine Learning	✅	❌	❌	❌
Anomaly Detection	✅	❌	❌	❌

Main packages



Main packages

```
# Install required packages (do once)
install.packages(c("tidyverse", "fpp3"))
```

```
# At the start of each session
library(fpp3)
```

Data sets in the FPP3 Package

Item	class	dim	Title
aus_accommodation	data.frame	592x5	Australian accommodation data
aus_airpassengers	data.frame	47x2	Air Transport Passengers Australia
aus_arrivals	data.frame	508x3	International Arrivals to Australia
aus_births	data.frame	4512x3	Australian births data
aus_fertility	data.frame	15120x4	Australian fertility rates
aus_inbound	data.frame	20748x4	Monthly short term (<1 year) visitor arrivals to Australia
aus_migration	data.frame	1360x3	Australian migration data
aus_mortality	data.frame	7740x5	Australian mortality data
aus_outbound	data.frame	11700x4	Monthly short term (<1 year) resident departures in Australia
aus_tobacco	data.frame	1232x3	Australian cigarette and tobacco expenditure
aus_vehicle_sales	data.frame	864x3	Australian vehicle sales
bank_calls	data.frame	27716x2	Call volume for a large North American commercial bank
boston_marathon	data.frame	265x5	Boston marathon winning times since 1897
canadian_gas	data.frame	542x2	Monthly Canadian gas production
guinea_rice	data.frame	42x2	Rice production (Guinea)
insurance	data.frame	40x3	Insurance quotations and advertising expenditure
melb_walkers	data.frame	1976x2	Average daily total pedestrian count in Melbourne
nsw_offences	data.frame	7308x3	Monthly offences in NSW
ny_childcare	data.frame	412x2	New York childcare data
otexts_views	data.frame	1561x2	OTexts page views
prices	data.frame	198x7	Price series for various commodities

Data sets in the tsibbledata Package

Item	class	dim	Title
PBS	data.frame	67596x9	Monthly Medicare Australia prescription data
ansett	data.frame	7407x4	Passenger numbers on Ansett airline flights
aus_livestock	data.frame	29364x4	Australian livestock slaughter
aus_production	data.frame	218x7	Quarterly production of selected commodities in Australia.
aus_retail	data.frame	64532x5	Australian retail trade turnover
gafa_stock	data.frame	5032x8	GAFA stock prices
global_economy	data.frame	15150x6	Global economic indicators
hh_budget	data.frame	88x8	Household budget characteristics
nyc_bikes	data.frame	4268x12	NYC Citi Bike trips
olympic_running	data.frame	312x4	Fastest running times for Olympic races
pelt	data.frame	91x3	Pelt trading records
vic_elec	data.frame	52608x5	Half-hourly electricity demand for Victoria, Australia

`tsibble` objects

- A `tsibble` allows storage and manipulation of multiple time series in R.
- It contains:
  - An index: time information about the observation
  - Measured variable(s): numbers of interest
  - Key variable(s): optional unique identifiers for each series
- It works with tidyverse functions.
- Other classes: `ts`, `zoo`, `xts`, `irts`, `msts`, `tibbletime`, `timeSeries` etc.)

Working with `tsibble` objects

We can use the:

- `filter()` function to select rows.
- `select()` function to select columns.
- `summarise()` function to summarise over keys.
- `mutate()` function to create new variables.

The `tsibble` index

Time index variables can be created with these functions:

Frequency	Function
Annual	`start:end`
Quarterly	`yearquarter()`
Monthly	`yearmonth()`
Weekly	`yearweek()`
Daily	`as_date()`, `ymd()`
Sub-daily	`as_datetime()`

`tsibble` objects

```
global_economy
```

```
# A tsibble: 15,150 x 6 [1Y]
# Key:       Country [263]
    Year Country              GDP Imports Exports Population
   Index Key                  Measured variables
 1  1960 Afghanistan  537777811.    7.02    4.13    8996351
 2  1961 Afghanistan  548888896.    8.10    4.45    9166764
 3  1962 Afghanistan  546666678.    9.35    4.88    9345868
 4  1963 Afghanistan  751111191.   16.9     9.17    9533954
 5  1964 Afghanistan  800000044.   18.1     8.89    9731361
 6  1965 Afghanistan 1006666638.   21.4    11.3     9938414
 7  1966 Afghanistan 1399999967.   18.6     8.57   10152331
 8  1967 Afghanistan 1673333418.   14.2     6.77   10372630
 9  1968 Afghanistan 1373333367.   15.2     8.90   10604346
```

tsibble objects

```
tourism
```

```
# A tsibble: 24,320 x 5 [1Q]
# Key:       Region, State, Purpose [304]
   Quarter Region   State Purpose  Trips
     Index Keys                   Measure
 1 1998 Q1 Adelaide SA    Business  135.
 2 1998 Q2 Adelaide SA    Business  110.
 3 1998 Q3 Adelaide SA    Business  166.
 4 1998 Q4 Adelaide SA    Business  127.
 5 1999 Q1 Adelaide SA    Business  137.
 6 1999 Q2 Adelaide SA    Business  200.
 7 1999 Q3 Adelaide SA    Business  169.
 8 1999 Q4 Adelaide SA    Business  134.
 9 2000 Q1 Adelaide SA    Business  154.
10 2000 Q2 Adelaide SA    Business  169.
```

Domestic visitor nights in thousands by state/region and purpose.

The `tsibble` index

Example

```
mydata <- tsibble(
  year = 2015:2019,
  y = c(123, 39, 78, 52, 110),
  index = year
)
mydata
```

```
# A tsibble: 5 x 2 [1Y]
   year     y
  <int> <dbl>
1  2015   123
2  2016    39
3  2017    78
4  2018    52
5  2019   110
```

The `tsibble` index

Example

```
mydata <- tibble(
  year = 2015:2019,
  y = c(123, 39, 78, 52, 110)
) |>
  as_tsibble(index = year)
mydata
```

```
# A tsibble: 5 x 2 [1Y]
   year     y
  <int> <dbl>
1  2015   123
2  2016    39
3  2017    78
4  2018    52
5  2019   110
```

The `tsibble` index

For observations more frequent than once per year, we need to use a time class function on the index.

```
z <- tibble(Month = paste(2019, month.abb[1:5]), Observation = c(50, 23, 34, 30, 25))
z |>
  mutate(Month = yearmonth(Month)) |>
  as_tsibble(index = Month)
```

```
# A tsibble: 5 x 2 [1M]
     Month Observation
     <mth>       <dbl>
1 2019 Jan          50
2 2019 Feb          23
3 2019 Mar          34
4 2019 Apr          30
```

Read a csv file and convert to a tsibble

Australian prison population

```
prison <- readr::read_csv("data/prison_population.csv") |>
  mutate(Quarter = yearquarter(date))
```

```
# A tibble: 3,072 x 7
   date       state gender legal     indigenous count Quarter
   <date>     <chr> <chr>  <chr>     <chr>      <dbl>   <qtr>
 1 2005-03-01 ACT   Female Remanded  ATSI           0 2005 Q1
 2 2005-03-01 ACT   Female Remanded  Other          2 2005 Q1
 3 2005-03-01 ACT   Female Sentenced ATSI           0 2005 Q1
 4 2005-03-01 ACT   Female Sentenced Other          0 2005 Q1
 5 2005-03-01 ACT   Male   Remanded  ATSI           7 2005 Q1
 6 2005-03-01 ACT   Male   Remanded  Other         58 2005 Q1
 7 2005-03-01 ACT   Male   Sentenced ATSI           0 2005 Q1
 8 2005-03-01 ACT   Male   Sentenced Other          0 2005 Q1
 9 2005-03-01 NSW   Female Remanded  ATSI          51 2005 Q1
```

Read a csv file and convert to a tsibble

```
prison <- readr::read_csv("data/prison_population.csv") |>
  mutate(Quarter = yearquarter(date)) |>
  select(-date) |>
  as_tsibble(
    index = Quarter,
    key = c(state, gender, legal, indigenous)
  )
```

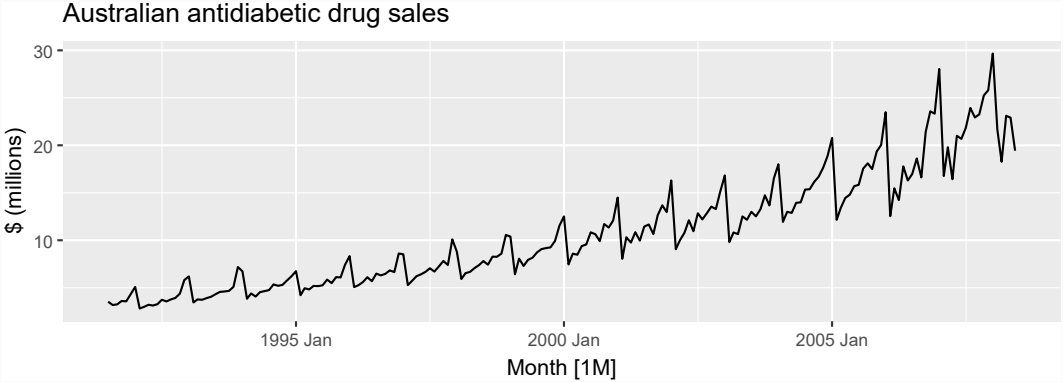
```
# A tsibble: 3,072 x 6 [1Q]
# Key:       state, gender, legal, indigenous [64]
   state gender legal    indigenous count Quarter
   <chr> <chr>  <chr>    <chr>      <dbl>   <qtr>
 1 ACT   Female Remanded ATSI           0 2005 Q1
 2 ACT   Female Remanded ATSI           1 2005 Q2
 3 ACT   Female Remanded ATSI           0 2005 Q3
 4 ACT   Female Remanded ATSI           0 2005 Q4
 5 ACT   Female Remanded ATSI           1 2006 Q1
```

Graphics

- Plots allow us to identify:
  - Patterns;
  - Unusual observations;
  - Changes over time;
  - Relationships between variables.

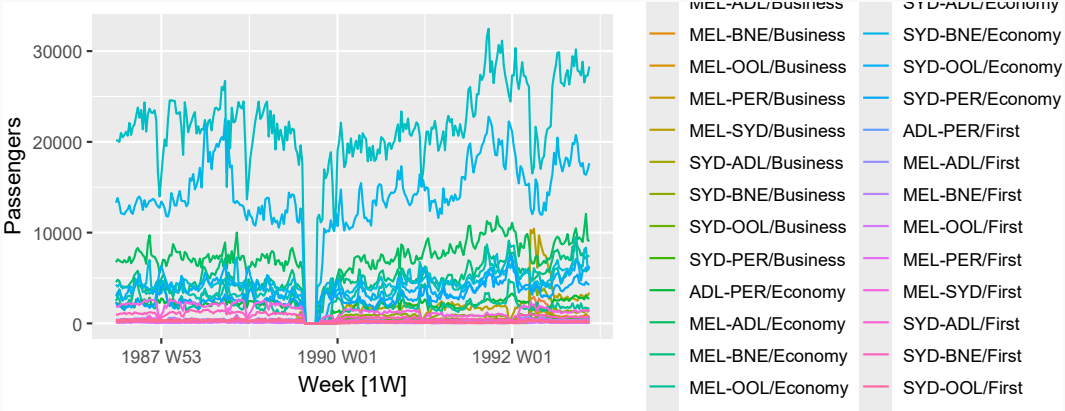
Time plots

```
a10 |>
  autoplot(Cost) +
  labs(y = "$ (millions)", title = "Australian antidiabetic drug sales")
```



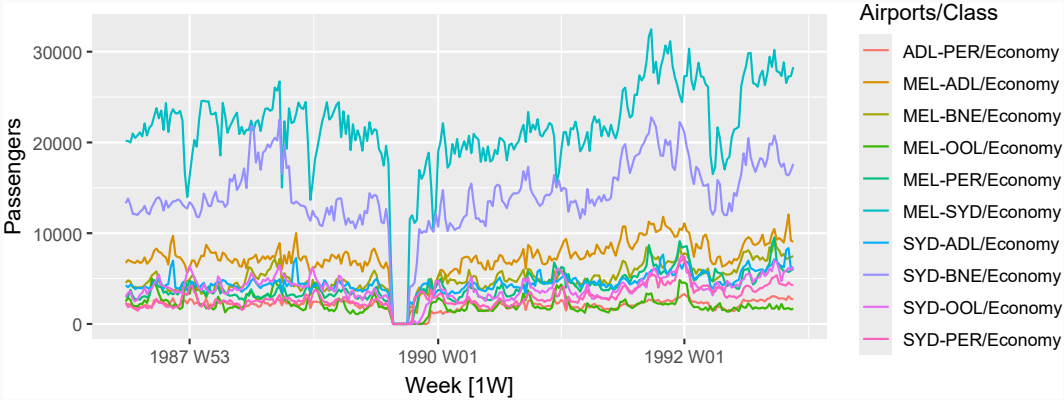
Ansett airlines

```
ansett |>
  autoplot(Passengers)
```



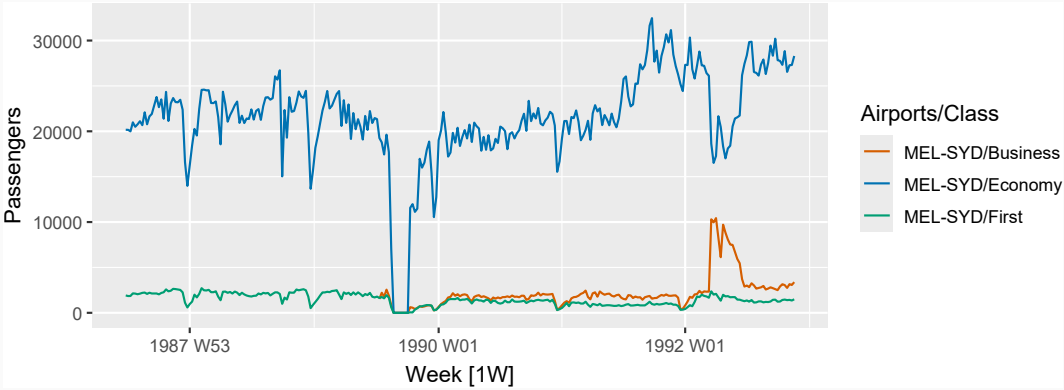
Ansett airlines

```
ansett |>
  filter(Class == "Economy") |>
  autoplot(Passengers)
```



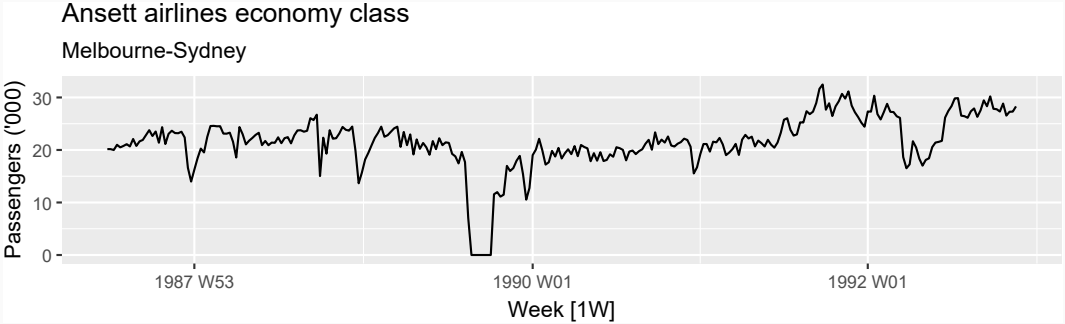
Ansett airlines

```
ansett |>
  filter(Airports == "MEL-SYD") |>
  autoplot(Passengers)
```



Ansett airlines

```
ansett |>
  filter(Airports == "MEL-SYD", Class == "Economy") |>
  mutate(Passengers = Passengers/1000) |>
  autoplot(Passengers) +
  labs(title = "Ansett airlines economy class", subtitle = "Melbourne-Sydney",
       y = "Passengers ('000)")
```



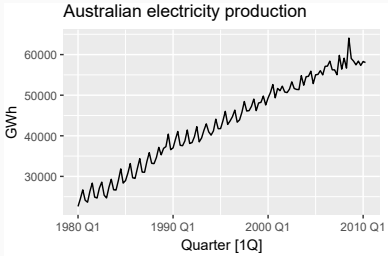
Recall: Time series patterns

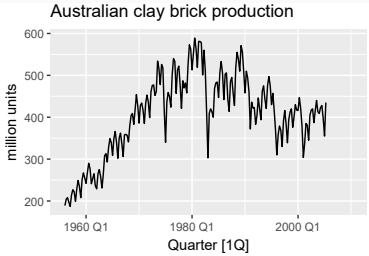
Trend pattern exists when there is a long-term increase or decrease in the data.

Seasonal pattern exists when a series is influenced by seasonal factors (e.g., the quarter of the year, the month, or day of the week).

Cyclic pattern exists when data exhibit rises and falls that are *not of fixed period* (duration usually of at least 2 years).

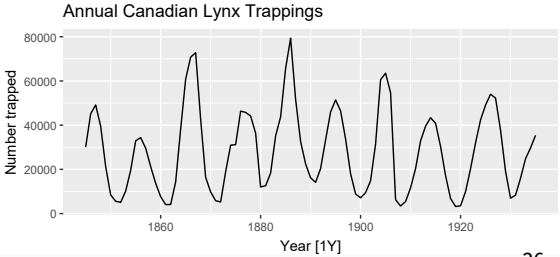
Time series patterns











Seasonal or cyclic?

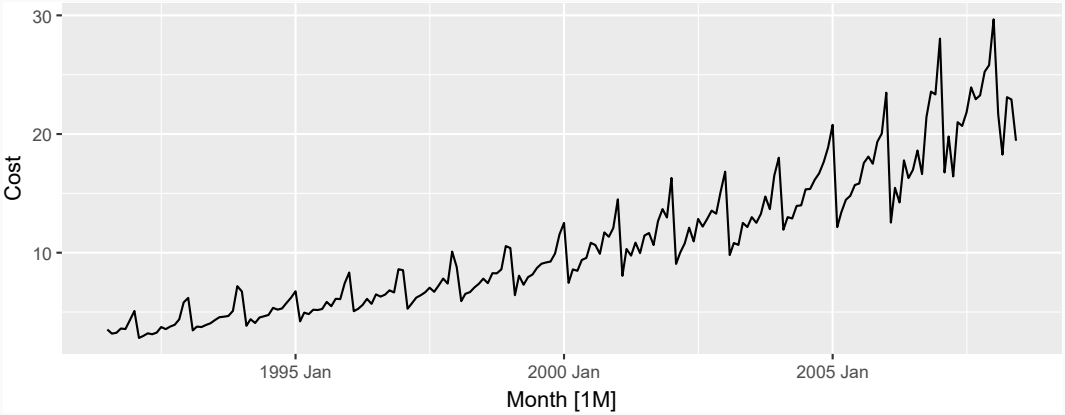
Differences between seasonal and cyclic patterns:

- seasonal pattern constant length; cyclic pattern variable length
- average length of cycle longer than length of seasonal pattern
- magnitude of cycle more variable than magnitude of seasonal pattern

The timing of peaks and troughs is predictable with seasonal data, but unpredictable in the long term with cyclic data.

Reminder: a10 time plot

```
a10 |>
  autoplot(Cost)
```



Seasonal plots

- Data plotted against the individual “seasons” in which the data were observed. (In this case a “season” is a month.)
- Something like a time plot except that the data from each season are overlapped.
- Enables the underlying seasonal pattern to be seen more clearly, and also allows any substantial departures from the seasonal pattern to be easily identified.
- In R: `gg_season()`
- Check for multiple seasonal periods

Seasonal subseries plots

- Data for each season collected together in time plot as separate time series.
- Enables the underlying seasonal pattern to be seen clearly, and changes in seasonality over time to be visualized.
- In R: `gg_subseries()`

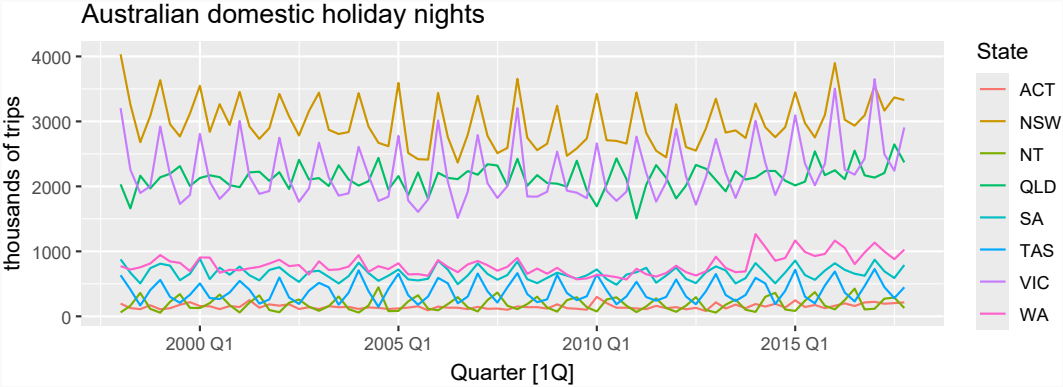
Australian holidays

```
holidays <- tourism |>
  filter(Purpose == "Holiday") |>
  group_by(State) |>
  summarise(Trips = sum(Trips))
```

```
# A tsibble: 640 x 3 [1Q]
# Key:       State [8]
   State Quarter Trips
   <chr>   <qtr> <dbl>
 1 ACT   1998 Q1  196.
 2 ACT   1998 Q2  127.
 3 ACT   1998 Q3  111.
 4 ACT   1998 Q4  170.
 5 ACT   1999 Q1  108.
 6 ACT   1999 Q2  125.
 7 ACT   1999 Q3  178.
 8 ACT   1999 Q4  218.
 9 ACT   2000 Q1  158.
```

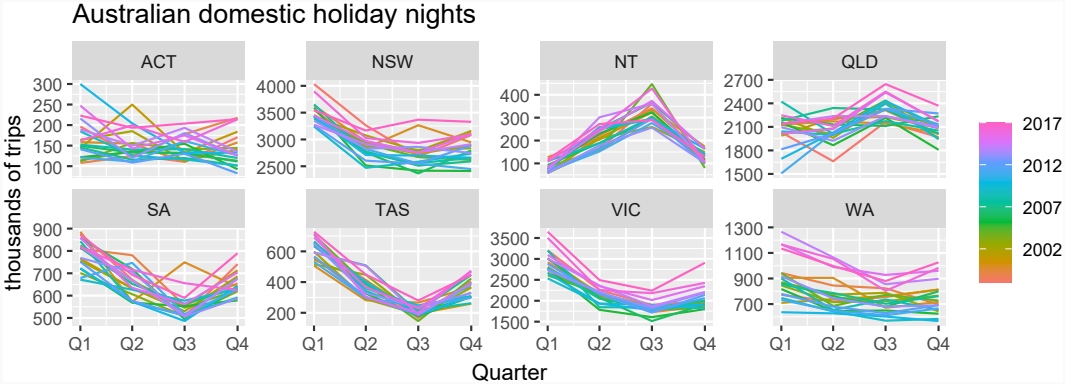
Australian holidays

```
holidays |> autoplot(Trips) +
  labs(y = "thousands of trips", title = "Australian domestic holiday nights")
```



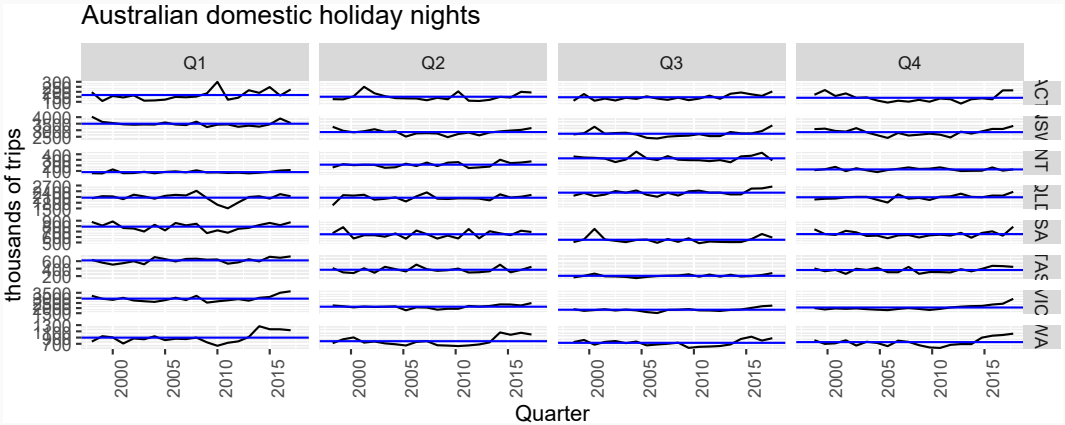
Seasonal plots

```
holidays |> gg_season(Trips) +
   facet_wrap(vars(State), nrow = 2, scales = "free_y")+
   labs(y = "thousands of trips", title = "Australian domestic holiday nights")
```



Seasonal subseries plots

```
holidays |>
  gg_subseries(Trips) +
  labs(y = "thousands of trips", title = "Australian domestic holiday nights")
```

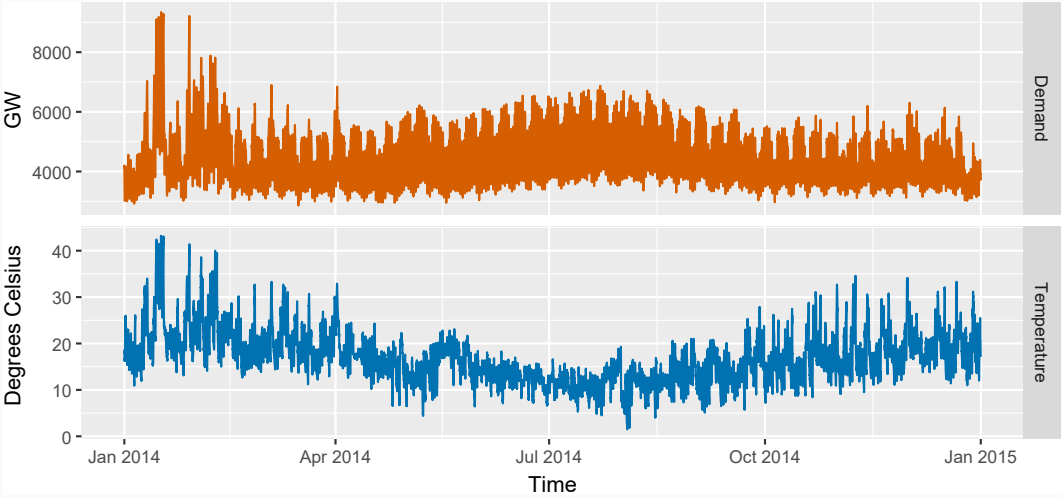


Electricity Demand in Victoria, Australia

```
vic_elec_day_type <- vic_elec |>
   filter(year(Time) == 2014) |>
   mutate(Day_Type = case_when(
      Holiday ~ "Holiday",
      wday(Date) %in% 2:6 ~ "Weekday",
      TRUE ~ "Weekend"))
vic_elec_day_type
```

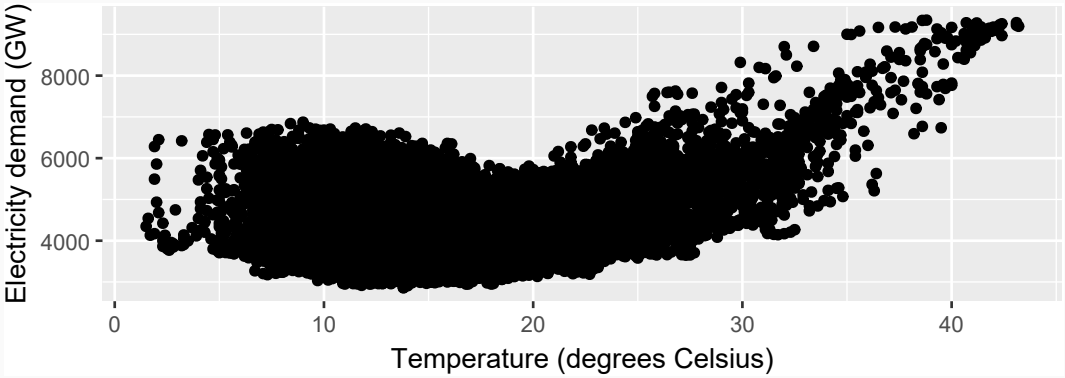
```
# A tsibble: 17,520 x 6 [30m] <Australia/Melbourne>
   Time                Demand Temperature Date       Holiday Day_Type
   <dttm>               <dbl>       <dbl> <date>     <lgl>   <chr>
 1 2014-01-01 00:00:00  4092.        18.7 2014-01-01 TRUE    Holiday
 2 2014-01-01 00:30:00  4198.        18.1 2014-01-01 TRUE    Holiday
 3 2014-01-01 01:00:00  3915.        18.2 2014-01-01 TRUE    Holiday
 4 2014-01-01 01:30:00  3673.        17.9 2014-01-01 TRUE    Holiday
 5 2014-01-01 02:00:00  3498.        17.6 2014-01-01 TRUE    Holiday
 6 2014-01-01 02:30:00  3339.        16.8 2014-01-01 TRUE    Holiday
 7 2014-01-01 03:00:00  3204.        16.3 2014-01-01 TRUE    Holiday
 8 2014-01-01 03:30:00  3100.        16.6 2014-01-01 TRUE    Holiday
```

Time plots


GW
8000
6000
4000
Demand
Degrees Celsius
40
30
20
10
0
Temperature
Jan 2014
Apr 2014
Jul 2014
Oct 2014
Jan 2015
Time

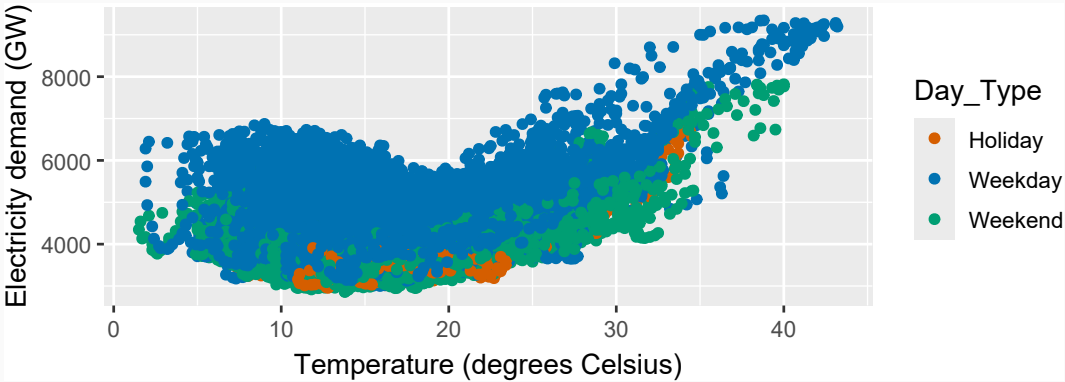
Scatterplots

```
vic_elec_day_type |>
  ggplot(aes(x = Temperature, y = Demand)) +
  geom_point() +
   labs(x = "Temperature (degrees Celsius)", y = "Electricity demand (GW)")
```



Scatterplots

```
vic_elec_day_type |>
  ggplot(aes(x = Temperature, y = Demand, colour = Day_Type)) +
  geom_point() +
   labs(x = "Temperature (degrees Celsius)", y = "Electricity demand (GW)")
```

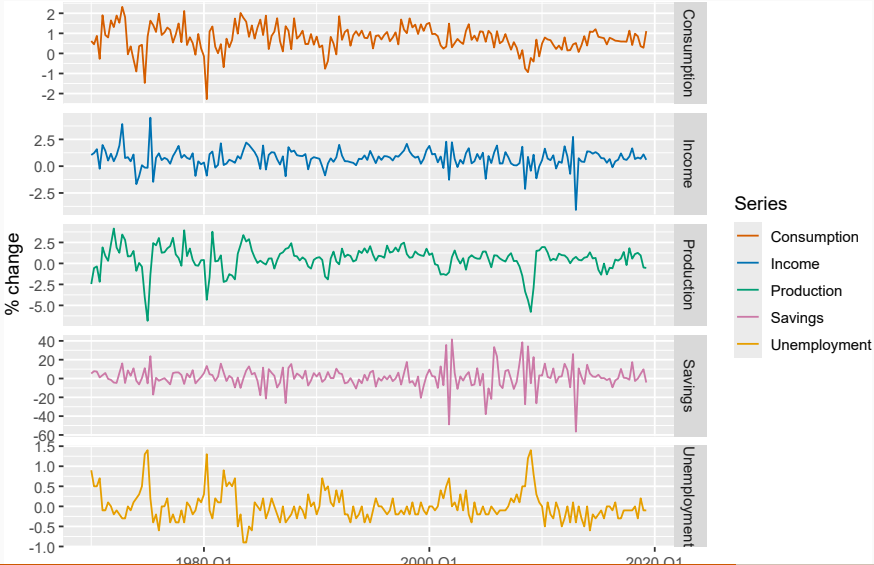


Correlation coefficient

Measures the extent of a linear relationship between two variables (y and x).

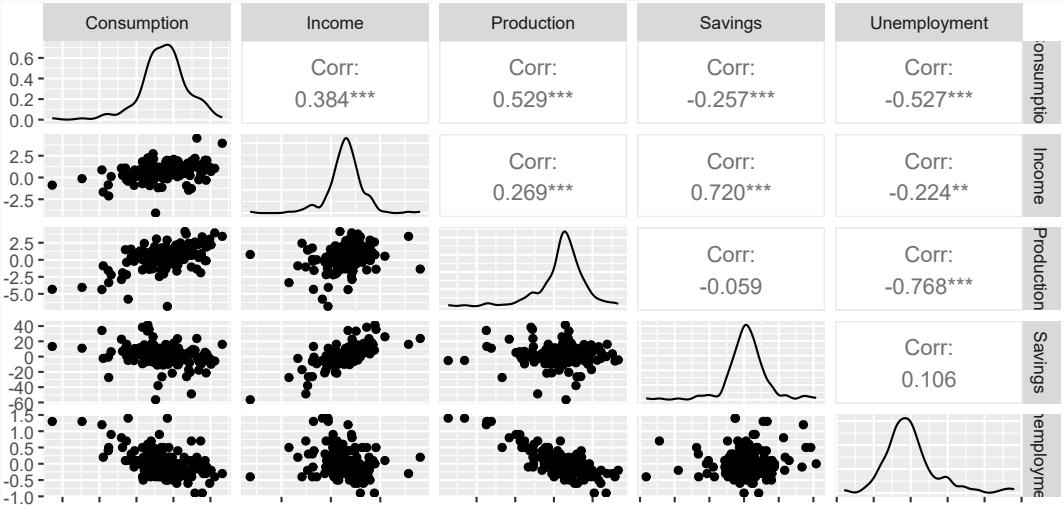
- $r = \frac{\sum_{t=1}^{T}(y_t-\bar{y})(x_t-\bar{x})}{\sqrt{\sum_{t=1}^{T}(y_t-\bar{y})^2}\sqrt{\sum_{t=1}^{T}(x_t-\bar{x})^2}}$
- Lies between -1 and 1

US consumption expenditure



US consumption expenditure

```
us_change |> GGally::ggpairs(columns = 2:6)
```



Lag and Lead

```
x <- 1:5
tibble(`behind (lag)` = lag(x), `x (Actual)`=x, `ahead (lead)` = lead(x)) |>
  knitr::kable(booktabs = TRUE)
```

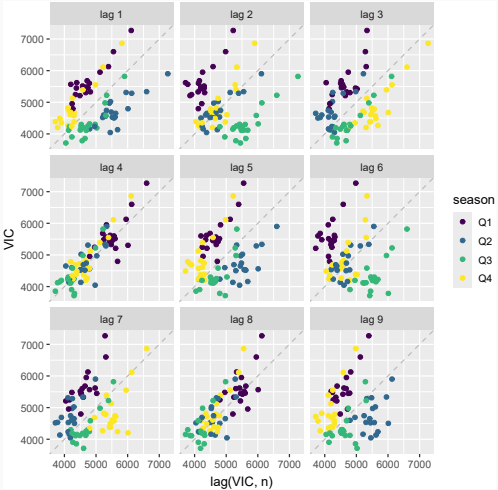
behind (lag)	x (Actual)	ahead (lead)
NA	1	2
1	2	3
2	3	4
3	4	5
4	5	NA

Example: Tourism

```
visitors <- tourism  |>
  group_by(State)  |>
  summarise(Trips = sum(Trips))  |>
  pivot_wider(values_from=Trips, names_from=State) |>
  select("VIC")
```

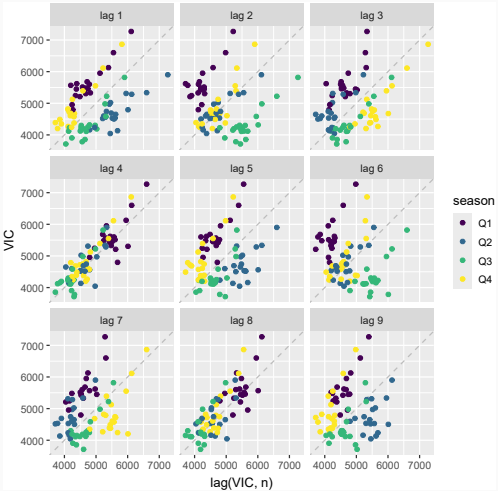
Example: Tourism

```
visitors |> gg_lag(geom = "point")
```



Example: Tourism

```
visitors |> gg_lag(geom = "point")
```



- Each graph shows y_t plotted against y_{t-k} for different values of k.
- The autocorrelations are the correlations associated with these scatterplots.
- r_1 = Correlation(y_t, y_{t-1})
- r_2 = Correlation(y_t, y_{t-2})
- r_3 = Correlation(y_t, y_{t-3})

$\vdots$

Autocorrelation

We denote the sample autocovariance at lag k by c_k and the sample autocorrelation at lag k by r_k. Then define

$$c_k = \frac{1}{T}\sum_{t=k+1}^{T}(y_t - \bar{y})(y_{t-k} - \bar{y})$$

and $$r_k = c_k/c_0$$

- r_1 indicates how successive values of y relate to each other
- r_2 indicates how y values two periods apart relate to each other
- r_k is *almost* the same as the sample correlation between y_t and y_{t-k}.

Autocorrelation

Results for first 9 lags for beer data:

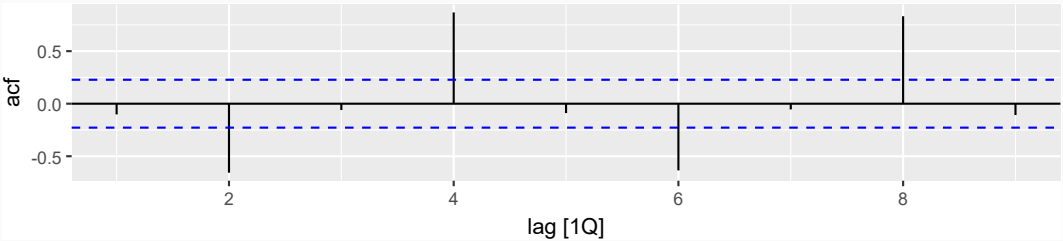
```
new_production |> ACF(Beer, lag_max = 9)
```

```
# A tsibble: 9 x 2 [1Q]
       lag     acf
  <cf_lag>   <dbl>
1       1Q -0.102
2       2Q -0.657
3       3Q -0.0603
4       4Q  0.869
5       5Q -0.0892
6       6Q -0.635
7       7Q -0.0542
```

Autocorrelation

Results for first 9 lags for beer data:

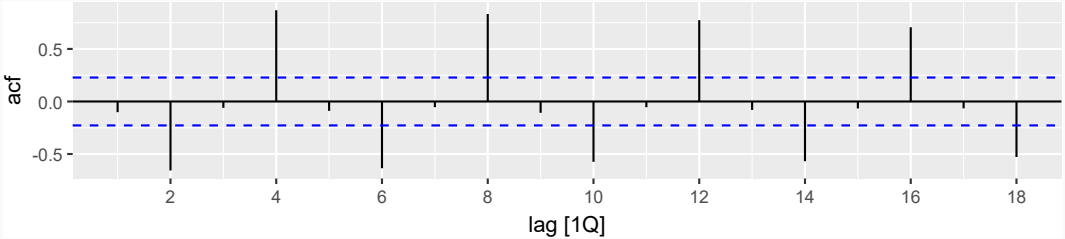
```
new_production |> ACF(Beer, lag_max = 9) |> autoplot()
```



- Together, the autocorrelations at lags 1, 2, ..., make up the *autocorrelation* or ACF.
- The plot is known as a **correlogram**

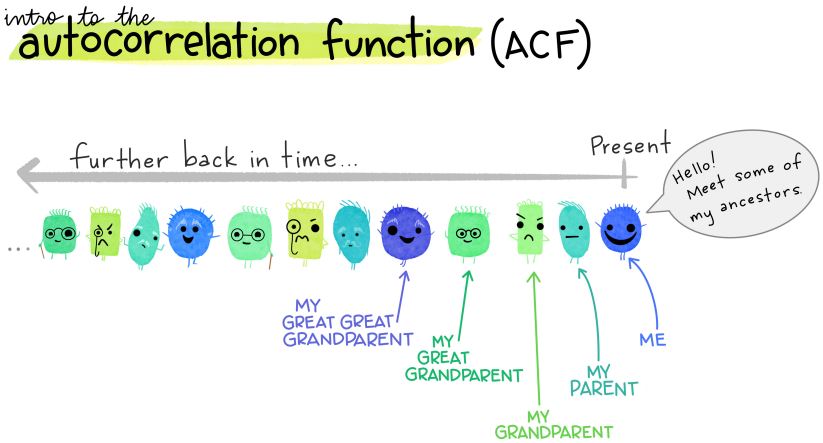
Autocorrelation

```
new_production |> ACF(Beer) |> autoplot()
```



- r_4 higher than for the other lags due to **the seasonal pattern in the data**: peaks tend to be **4 quarters** apart and troughs tend to be **4 quarters** apart.
- r_2 is more negative than for the other lags because troughs tend to be 2 quarters behind peaks.

Autocorrelation functions



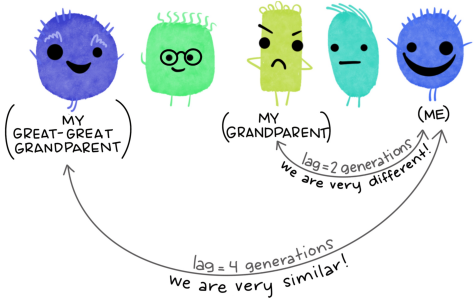
Artwork by @allison_horst

Autocorrelation functions

in our family MONSTERS tend to be...

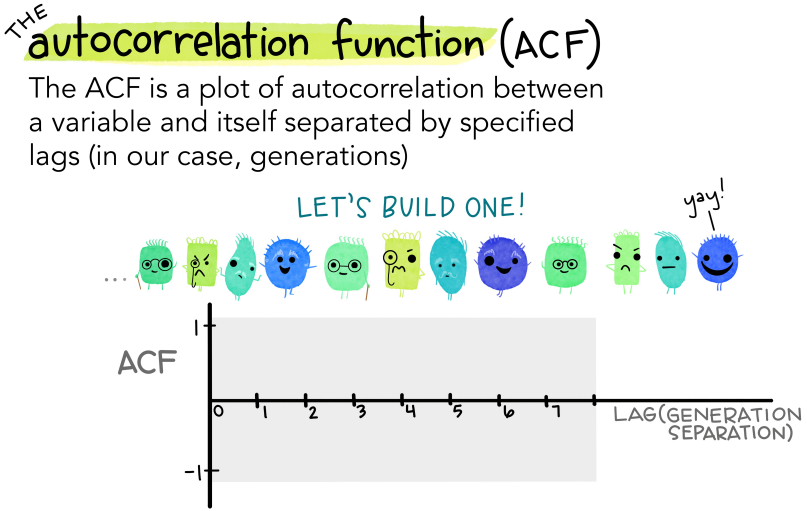
- A little similar to their parent and great-grandparent
- Very different from their grandparent
- Very similar to their great-great grandparent

FOR EXAMPLE:



Artwork by @allison_horst

Autocorrelation functions

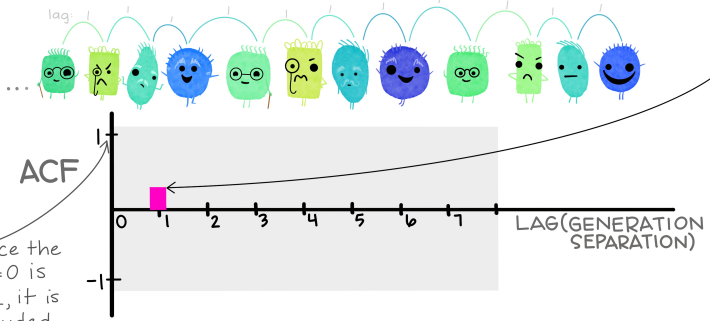


Artwork by @allison_horst

Autocorrelation functions

At lag =1, we find the correlation between **monsters** and their **parent.**

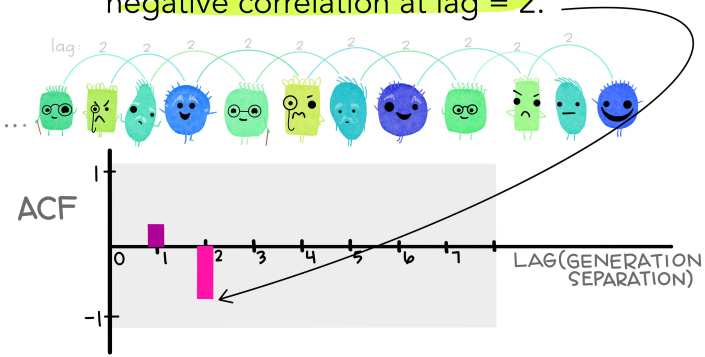
They are somewhat positively correlated.



Artwork by @allison_horst

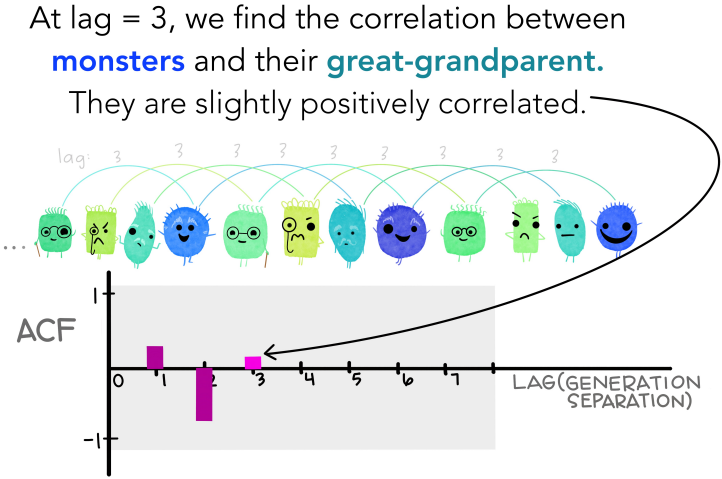
Autocorrelation functions

At lag = 2, we find the correlation between **monsters** and their **grandparent**.
Since they tend to be very different, we find a negative correlation at lag = 2.



Artwork by @allison_horst

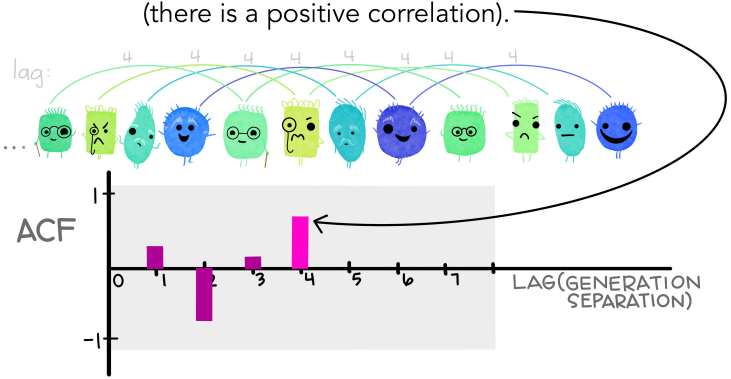
Autocorrelation functions



Artwork by @allison_horst

Autocorrelation functions

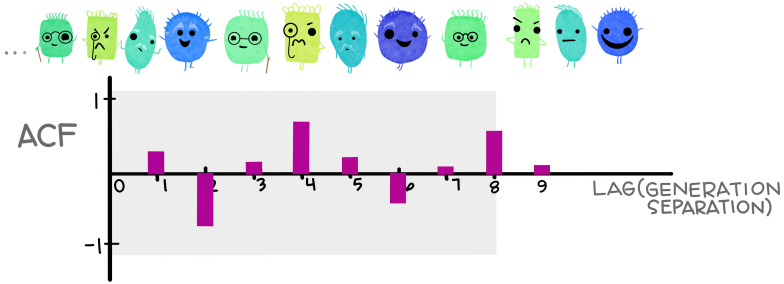
At lag = 4, we find the correlation between **monsters** and their **great-great grandparent**. They tend to be very similar (there is a positive correlation).



Artwork by @allison_horst

Autocorrelation functions

...and we continue finding the correlations as we increase the lag (generations) between the monsters...

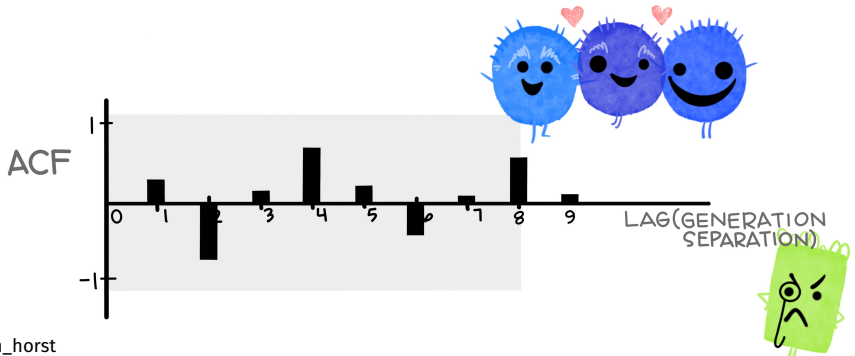


Artwork by @allison_horst

Autocorrelation functions

in Summary:

The autocorrelation function (ACF) tells us the correlation between observations and those that came before them, separated by different lags (here, monster generations)!

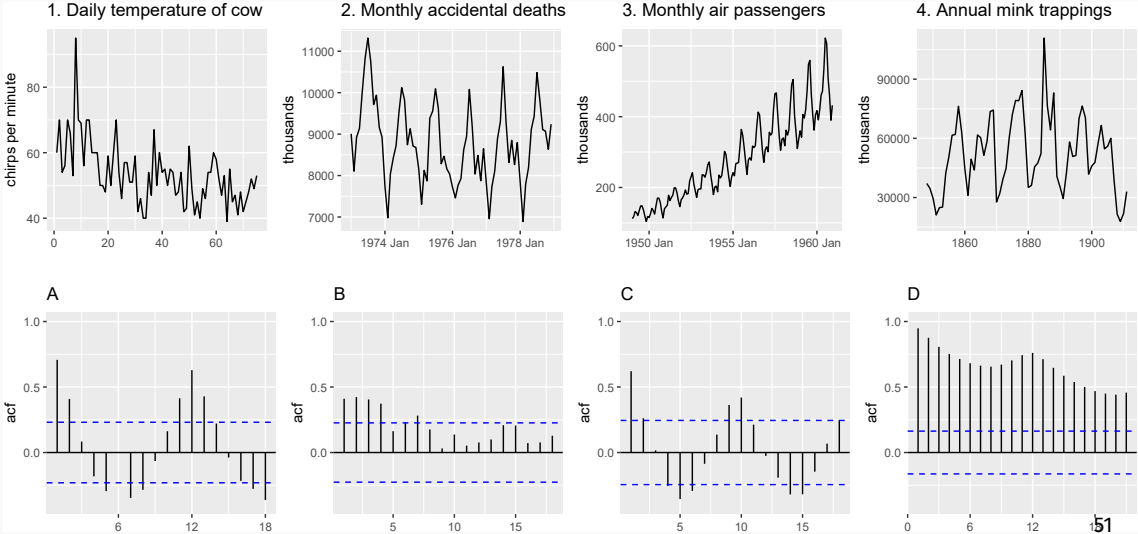


Artwork by @allison_horst

Trend and seasonality in ACF plots

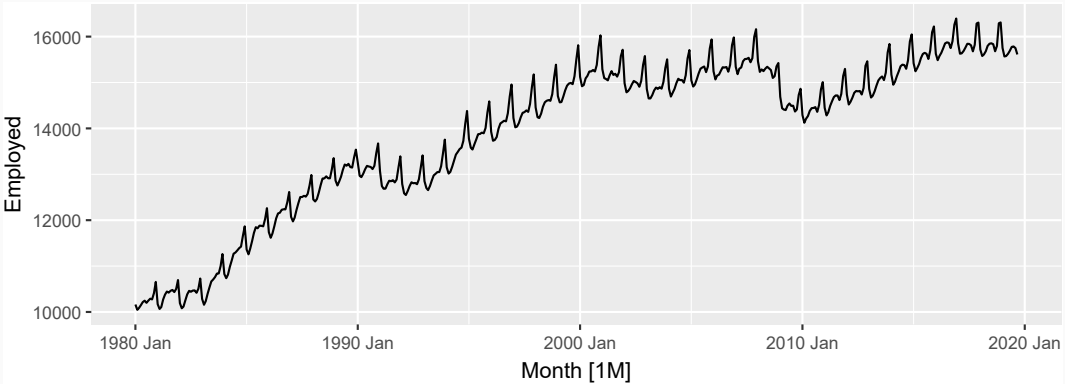
- When data have a trend, the autocorrelations for small lags tend to be large and positive.
- When data are seasonal, the autocorrelations will be larger at the seasonal lags (i.e., at multiples of the seasonal frequency)
- When data are trended and seasonal, you see a combination of these effects.

Which is which?



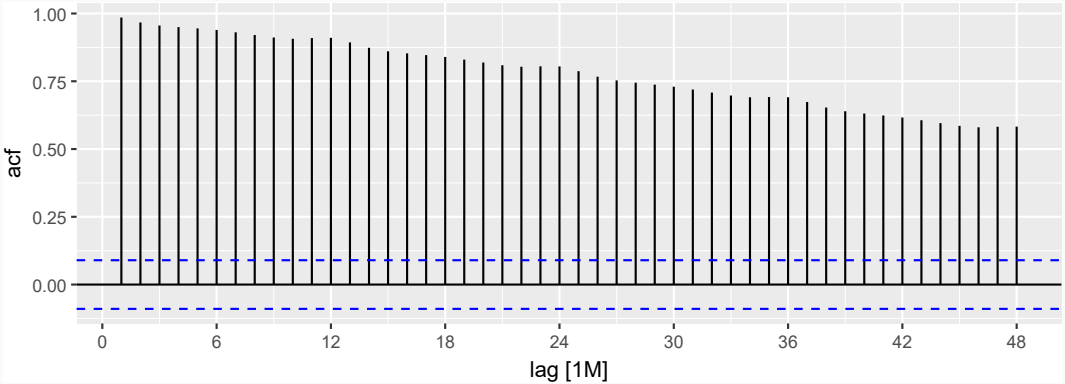
US retail trade employment

```
retail <- us_employment |>
  filter(Title == "Retail Trade", year(Month) >= 1980)
retail |> autoplot(Employed)
```



US retail trade employment

```
retail |>
  ACF(Employed, lag_max = 48) |>
  autoplot()
```



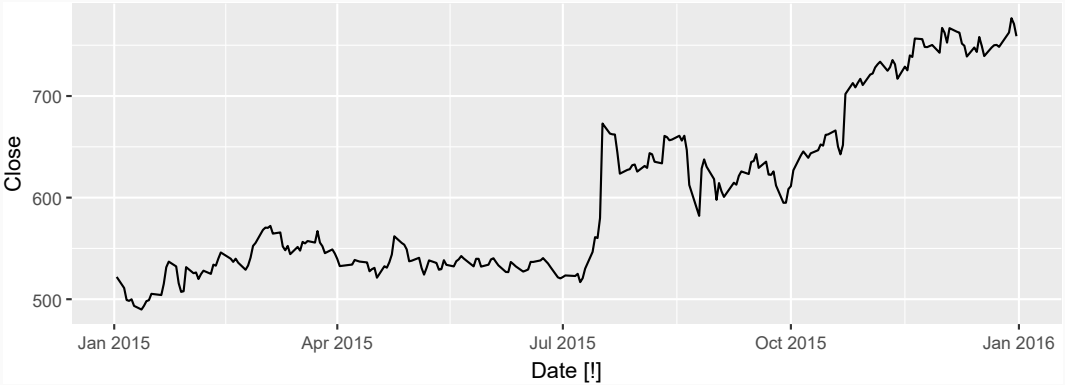
Google stock price

```
google_2015 <- gafa_stock |>
  filter(Symbol == "GOOG", year(Date) == 2015) |>
  select(Date, Close)
google_2015
```

```
# A tsibble: 252 x 2 [!]
   Date       Close
   <date>     <dbl>
 1 2015-01-02  522.
 2 2015-01-05  511.
 3 2015-01-06  499.
 4 2015-01-07  498.
 5 2015-01-08  500.
 6 2015-01-09  493.
 7 2015-01-12  490.
 8 2015-01-13  493.
 9 2015-01-14  498.
10 2015-01-15  499.
```

Google stock price

```
google_2015 |> autoplot(Close)
```



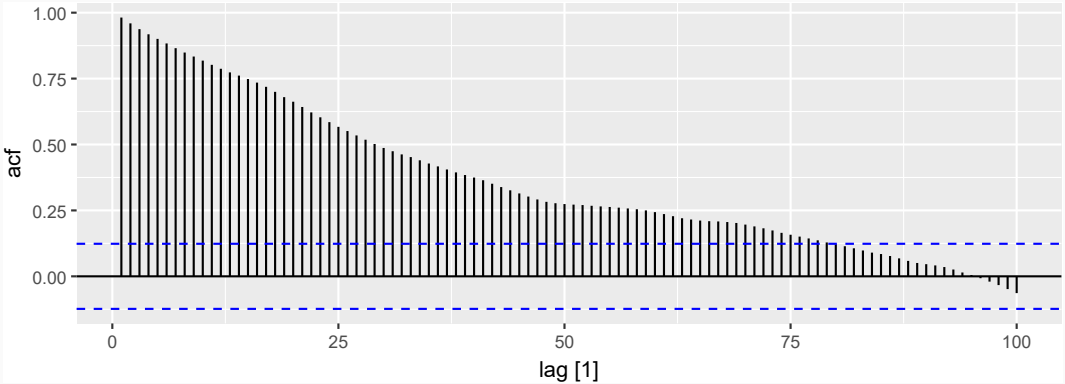
Google stock price

```
google_2015 |>
  ACF(Close, lag_max = 100)
```

```
# A tsibble: 100 x 2 [1]
        lag   acf
   <cf_lag> <dbl>
 1        1 0.982
 2        2 0.959
 3        3 0.937
 4        4 0.918
 5        5 0.901
 6        6 0.883
 7        7 0.865
 8        8 0.849
 9        9 0.834
10       10 0.818
# i 90 more rows
```

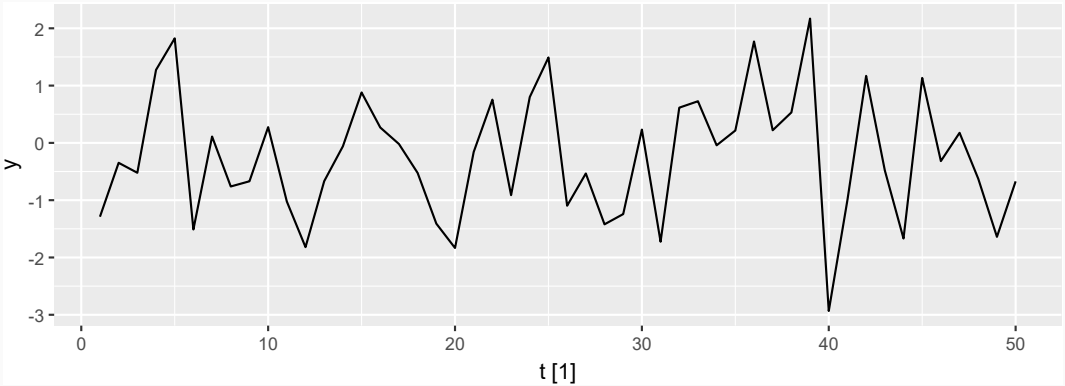
Google stock price

```
google_2015 |>
  ACF(Close, lag_max = 100) |>
  autoplot()
```



Example: White noise

```
set.seed(30)
wn <- tsibble(t = 1:50, y = rnorm(50), index = t)
wn |> autoplot(y)
```



Example: White noise

```
set.seed(30)
wn <- tsibble(t = 1:50, y = rnorm(50), index = t)
wn |> autoplot(y)
```

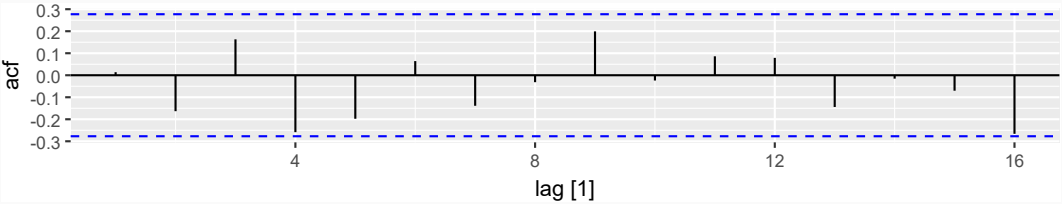


White noise data is uncorrelated across time with zero mean and constant variance.
(Technically, we require independence as well.)

Example: White noise

```
wn |> ACF(y)
```

r_1	r_2	r_3	r_4	r_5	r_6	r_7	r_8	r_9	r_{10}
0.014	-0.163	0.163	-0.259	-0.198	0.064	-0.139	-0.032	0.199	-0.024



- Sample autocorrelations for white noise series.
- Expect each autocorrelation to be close to zero.
- Blue lines show 95% critical values.

Sampling distribution of autocorrelations

Sampling distribution of r_k for white noise data is asymptotically N(0,1/T).

- 95% of all r_k for white noise must lie within $\pm 1.96/\sqrt{T}$.
- If this is not the case, the series is probably not WN.
- Common to plot lines at $\pm 1.96/\sqrt{T}$ when plotting ACF. These are the **critical values**.